

《面向高利害考试的试卷级 CAT：基于多目标优化》(PCAT) 精读笔记

原文: pcat-paper-level-cat-moo_8.pdf(Li et al., KDD '25, 华东师大 ECNU-ILOG 组) · 代码: <https://github.com/ECNU-ILOG/PCAT> (已 clone 到本目录 PCAT/) · 笔记于 2026-06-26

这篇是把 CAT「换赛道」的工作: 传统 CAT 是一道一道实时选题 (题目级), 这篇改成一卷一卷分阶段发卷 (试卷级), 并把组卷当成一个多目标优化 (MOO) 问题用 NSGA-II 离线解出来。动机不是「估得更准」, 而是面向 GRE/GMAT 这类高利害标准化考试的三个现实痛点: 可比性、成本、多因素平衡。下面按规范展开。

一、解决什么问题

先看传统题目级 CAT 的循环: CDM 估能力 $\rightarrow$ 选题算法挑下一题 $\rightarrow$ 学生答 $\rightarrow$ CDM 更新 $\dots$ 每个学生走出一条独一无二的答题路径 (论文叫 one-for-each「一人一卷」范式)。这在练习场景很好, 但搬到 GRE/GMAT 这种高利害考试就有三个硬伤:

1. **缺乏可比性**: 每个考生题目都不一样, 没法横向比较谁强谁弱, 考试结果的透明度和说服力大打折扣——高利害考试最忌讳这个。
2. **实施成本高**: 依赖实时交互, 每答一题就要在线更新 CDM、重新选题。考试拉长, 还要维护一套高并发的在线 CAT 系统; 网络/设备一抖动就出问题; GRE 还要频繁开考, 成本爆炸。
3. **多因素难平衡**: 传统 CAT 只盯着「估得准、估得快」, 不太管属性覆盖 (题目考的知识点全不全) 和题目曝光 (同一道题被太多人做会泄题, 威胁题库安全)。

已有的 MOO+CAT 工作 (MOOIT、GMOCAT) 虽然也用多目标, 但还是题目级、还是实时、还是一人一卷, 没解决可比性和成本问题。组卷 (test assembly) 领域的并行组卷又是非自适应的。所以「既自适应、又可比、又低成本、又多目标平衡」这个组合, 是空白。

PCAT 的思路 (看图 1c): 把一场考试切成几个阶段, 每阶段发一份整卷。第一阶段所有人做同一份卷; 后面阶段根据你前面的表现, 自适应地给你发难卷或易卷。关键是——所有计算 (CDM 诊断、组卷) 都在考前离线做完, 考试时只是「按规则发卷」, 零实时计算。而且用基于种群的 NSGA-II, 一次跑出一整组帕累托最优试卷, 管理者可以按需挑 (人多就挑曝光低的那套)。

二、问题定义 (输入 / 输出)

符号: - 题库 $Q = \{Q_1, \dots, Q_n\}$, n 道题。- 属性集 $\mathcal{C} = \{C_1, \dots, C_d\}$ (即知识概念), d 个。- Q -matrix $\in \{0, 1\}^{n \times d}$: 第 j 行是题 Q_j 考查哪些属性的 0/1 向量 (教育领域常说的 Q 矩阵)。- 作答日志 $\mathcal{R}$: 三元组 (S_i, Q_j, r_{ij}) 的集合, $r_{ij} \in \{0, 1\}$ 答对/错。

输入: 题库 Q + Q 矩阵 + 一批学生的作答日志 $\mathcal{R}$ 。输出: 一组试卷集 $\mathcal{P} = \{P_1, \dots, P_m\}$ (种群里每个个体 P_i 是一套完整的 k 阶段考卷方案)。

一套试卷 P_i 长什么样 (本文用受 GRE 启发的 2 阶段、3 难度设定): 假设 $|P_i| = 30$ 道题, 会被切成 3 份各 10 题的卷子: 易卷、中卷、难卷。
- 阶段 1: 所有人做中卷 (10 题)。- 阶段 2: 前一阶段答得好的一半人 $\rightarrow$ 做难卷; 另一半 $\rightarrow$ 做易卷。- 每个学生总共做 $Step = \frac{2}{3}|P_i| = 20$ 题。

一个具体小例子: 考生小王, 阶段 1 做中卷 10 题答对 7 题 (正确率 0.7, 排在前 50%) $\rightarrow$ 阶段 2 系统自动发难卷, 他再做 10 题。全程系统不算任何东西, 只是查了一下「 $0.7 \geq 0.5$ 吗? 是 $\rightarrow$ 发难卷」。考完把这 20 题的作答喂给离线 CDM, 诊断出他的能力画像, 再在查询集上验证准不准。

评测的特殊处理 (很重要): 传统 CAT 评测要求「选中的题在日志里有作答记录」, 但 PCAT 试卷是预先组好的, 可能含某学生没做过的题 $\rightarrow$ 没法评测。所以本文先用启发式算法把稀疏日志稠密化成 $\mathcal{R}_D$ (每个学生对每道题都有记录, 本质是 NP 难的「最稠密子图」问题, 用贪心迭代删最稀疏的行/列来近似), 在稠密子集上才能模拟「发任意卷、收任意作答」。

三、模型结构 (PCAT = NSGA-II 套一个模拟考试的适应度函数)

PCAT 就是标准 NSGA-II 进化算法, 核心创新在解的编码和适应度怎么算。整体「输入 $\rightarrow$ 处理 $\rightarrow$ 输出」:

模块 1: 解的表示 (双层编码) ——核心创新 - 老办法: 一个解 = 长度 n 的 0-1 向量 (选哪些题)。题库一大就又稀疏又费内存, 还没教育含义。- PCAT: 一个解 = $|P_i| \times 2$ 的矩阵, 每行是一个 (类型 τ , 难度 δ) 对。- 类型 $\tau \in \{1, \dots, \eta\}$: 整数, 代表「考查哪个属性组合」 (比如 $\tau = 3$ 代表「同时考代数 + 几何」这一类题)。- 难度 $\delta \in \mathbb{R}$: 实数, 想要的难度。- 好处: 30 题只要 60 个数 ($2h$), 而不是 n (可能上万) 个数; 而且自带教育语义。

模块 2: 解码 (邻域投影) ——把抽象的 (类型, 难度) 对落地成题库里真实的题 (见公式 1)。

模块 3: 适应度评估 (模拟一场考试) 对每个候选解 P_i : 解码成真题 $\rightarrow$ 按难度排序切成易/中/难三卷 $\rightarrow$ 模拟 2 阶段自适应发卷 $\rightarrow$ 收集全体学生模拟作答 $\rightarrow$ 用 CDM 诊断 $\rightarrow$ 算出 4 个目标 AUC/ACC/COV/EXP。

模块 4: NSGA-II 进化: 随机初始化种群 → 交叉变异生成子代 → 用「非支配排序 + 拥挤距离」选出下一代 → 迭代 $T = 100$ 代 → 输出整个帕累托前沿种群。

数据流: 种群 (一堆 P_i 矩阵) → 解码 → 模拟考试打分 (4 目标) → 非支配选择 → 新种群 → → 帕累托前沿。

四、关键公式详解 (重点)

公式 1: 邻域投影解码

$$Q_i = \arg \min_{q \in Q_{\tau_i}} |\delta_q - \delta_i|$$

- Q_{τ_i} : 题库里类型为 τ_i 的所有题。 δ_q : 题 q 在题库里真实的难度 (来自 CDM 训练)。 δ_i : 解里想要的难度。
- 含义: 在「对的类型」里, 挑难度跟你想要的最接近的那道真题。
- 举例: 解里某行是 $(\tau = 3, \delta = 0.62)$, 意思「我要一道考代数 + 几何、难度约 0.62 的题」。题库里类型 3 的题难度有 $\{0.20, 0.55, 0.71, 0.90\}$ 。算 $|\delta_q - 0.62|$ 得 $\{0.42, 0.07, 0.09, 0.28\}$, 最小是 0.55 那道 (差 0.07) → 解码成它。这样抽象编码就变成了一道真实考题。
- 作用: 连接「连续优化空间」和「离散题库」的桥梁, 让进化算法能在紧凑空间里搜, 再投影回真题。

公式 2: 诊断质量 AUC / ACC

$$\text{AUC} = \int_0^1 \text{TPR}(f) \cdot \text{FPR}(f) df, \quad \text{ACC} = \frac{\text{TP} + \text{TN}}{\text{TP} + \text{TN} + \text{FP} + \text{FN}}$$

- 就是用「这套卷子诊断出的能力」去预测学生在留出题上的对错, 预测得越准说明卷子诊断质量越高。ACC 就是预测正确比例。
- 举例: 留出集 100 条作答, CDM 预测对了 78 条 (TP+TN=78) → ACC=0.78。论文里 PCAT 在 ASSIST 上 $\text{Step} = 20$ 的 ACC 做到 79.97%, 比次优高一截。
- 作用: MOO 的第 1、2 个目标 (要最大化)。注意原文 AUC 公式写成 $\text{TPR} \cdot \text{FPR}$ 的积分 (标准 AUC 应是 TPR 对 FPR 积分, 此处照录原文)。

公式 3 / 7: 属性覆盖 COV

$$\text{COV} = \frac{1}{|\text{ST}|} \sum_{s \in \text{ST}} \left(\frac{1}{|C|} \left| \bigcup_{q \in Q_s} Q\text{-matrix}(q) \right| \right)$$

- 对每个学生 s : 把他做过的所有题 Q_s 的 Q 矩阵**取并集** (覆盖了哪些知识点), 数一数有几个, 再除以总知识点数 $|C|$ 归一化; 最后对全体学生平均。
- 举例: 总共有 $|C| = 5$ 个知识点 $\{C_1..C_5\}$ 。小王做了 3 道题, 分别考 $\{C_1\}$ 、 $\{C_1, C_2\}$ 、 $\{C_3, C_4\}$, 并集 = $\{C_1, C_2, C_3, C_4\}$ 共 4 个 → 他的覆盖率 $4/5 = 0.8$ 。再对所有学生平均得 COV。
- 作用: MOO 第 3 个目标 (最大化)。COV 高 = 考得全面, 不会只盯一个知识点反复考。

公式 4 / 8: 题目曝光 EXP

$$\text{EXP} = \frac{1}{|\text{ST}|} \sum_{s \in \text{ST}} \left(\frac{\sum_{q \in Q_s} (N_q / |\mathcal{S}| - r)^2}{r} \right), \quad r = \text{Step} / n$$

- N_q : 题 q 在原始日志里被作答过的次数; $|\mathcal{S}|$: 学生总数; $N_q / |\mathcal{S}|$ 是题 q 的「历史曝光率」。 $r = \text{Step} / n$ 是「理想均匀曝光率」(一场考 Step 题、题库 n 题, 理想下每题被选概率)。
- 含义: 衡量每道被选中的题, 其曝光率偏离「理想均匀」有多远, 用方差形式。**EXP 越小越好**——说明选题分布越均匀, 没有少数几道题被反复曝光 (防泄题)。
- 举例: 题库 $n = 100$, $\text{Step} = 20$, 则理想曝光率 $r = 20/100 = 0.2$ 。某道被选的题历史曝光率 $N_q / |\mathcal{S}| = 0.5$ (被一半学生做过, 太热门), 偏差 $(0.5 - 0.2)^2 = 0.09$; 另一道冷门题 0.18, 偏差 $(0.18 - 0.2)^2 = 0.0004$ 。把学生选中题的这些偏差求和再除以 r 即得。热门题选得越多 EXP 越大越糟。
- 作用: MOO 第 4 个目标 (最小化)。直接优化它来保护题库安全。论文里 PCAT 的 EXP 最低。

公式 5: 变异

$$\tau' = \tau + \Delta\tau, \quad \delta' = \delta + \Delta\delta$$

- 对一个 (类型, 难度) 对做随机扰动: 类型跳到另一个合法类型, 难度在 $[\delta_{\min}, \delta_{\max}]$ 内挪一点。配合交叉 (类型用旅行商交叉、难度用两点交叉) 探索解空间。
- 举例: $(\tau = 3, \delta = 0.62)$ 变异成 $(\tau = 5, \delta = 0.70)$ ——换了考查的属性组合、难度调高一点, 产生一个新候选题位。
- 作用: 进化算法的探索算子, 保证种群多样性、跳出局部最优。

公式 6: 拥挤距离

$$d_j(P_i) = \frac{f_j(P_{i+1}) - f_j(P_{i-1})}{f_j^{\max} - f_j^{\min}}, \quad D(P_i) = \sum_{j=1}^o d_j(P_i)$$

- 在第 j 个目标上把解排序, 看解 P_i 左右两个邻居的目标值差距, 归一化; 对所有 $o = 4$ 个目标求和。
- 含义: $D(P_i)$ 大 = 这个解周围比较「空旷」(独特), NSGA-II 会优先保留它, 避免种群挤成一团、维持帕累托前沿的多样性。
- 举例: 在 EXP 目标上, P_i 的左邻 EXP=0.30、右邻=0.50, 该目标范围 $[0.1, 0.9]$, 则 $d_{EXP} = (0.50 - 0.30)/(0.9 - 0.1) = 0.25$ 。四个目标的 d 加起来就是总拥挤距离。两个解非支配等级相同时, 挑 D 大的那个进下一代。
- 作用: NSGA-II 的多样性维持机制, 让最终帕累托前沿铺开、覆盖各种权衡偏好。

五、代码实现对照

代码已 clone 在 PCAT/, 核心在 scripts/run_code/main.py, 基于 **pymoo** 的 NSGA2。论文公式 $\leftrightarrow$ 代码对应:

论文	代码位置	说明
NSGA-II 主框架 (算法 1)	main.py:242 algorithm = NSGA2(pop_size=100, ...)	pymoo 的 NSGA2, 注意代码里 pop_size=100 (论文正文写 80, 以论文为准, 复现看代码)
公式 5 交叉/变异	main.py:245-246 CustomCrossover(...), CustomMutation(..., probab_mutation=0.5)	自定义算子, 对 (类型, 难度) 双层编码分别处理
公式 1 邻域投影解码	main.py:260 def get_paper(x, paper_config) : 265-276	在 question_type[int(x[i])] 里找难度 (question_difficulty) 最接近的题
难度来自 CDM	main.py:211 question_difficulty = model_train.model.k_difficulty.weight...	用 NCDM 训练出的 k_difficulty 当题目难度 δ_q
三难度切卷	main.py:302-306 按难度排序后 paper1/paper2/paper3 = 易/中/难	sorted(question_list, key=lambda x: x[1]) 再三等分
自适应发卷 (算法 1 的 rank>0.5)	main.py:337 if p >= 0.5: → 发难卷 paper3, 否则易卷 paper1	p = 学生在中卷 paper2 上的正确率, 对应「百分位排名 > 0.5」
公式 4 EXP	main.py:332 overlap += (question_overlap[prob] - test_length/prob_num)**2	正是 $(N_q/ \mathcal{S} - r)^2$ 的累加, question_overlap 即 $N_q/ \mathcal{S} $
CDM	model/NCD.py (NeuralCD)	论文用 NCDM 当学生画像模块
基线策略	strategy/ 下 MFI/KLI/MAAT/BECAT/NCAT/BOBCAT_strategy.py	各对比方法
数据稠密化 (5.1.1)	scripts/dense_paper.py、generate_densedata.py	NP 难最稠密子图的贪心近似

读代码时最值得看的是 main.py 里 get_paper (解码) 和第 290-345 行那段模拟考试 + 算 COV/EXP 的循环——它就是「适应度函数」的实体, 把论文第 4.2 节文字逐行变成了代码。

六、小结

实验结论要点 (四数据集 ASSIST/MOOC/PROB-STA/COM-TH, $Step = 10/20$): - **诊断质量**: PCAT 的 AUC/ACC 普遍最优或接近最优, ACC 平均超 SOTA 约 5% (ASSIST 上 AUC 87.19% vs 次优 83.01%, 提升最明显)。作者解释: PCAT 是**全局搜索** (一次评估整套卷的总体效果), 而基线是**逐题局部贪心**, 所以全局期望更好。- **覆盖 & 曝光**: PCAT 的 COV 最高、EXP 最低, 因为这俩本就是显式优化目标。MOOC 上 EXP 大家相等 (该数据集曝光天然均匀, 指标失去区分度)。- **在线时间**: PCAT 近乎 0 (1~5s), 而 MAAT/BECAT 动辄几十分钟到 >7 小时——这是 PCAT 最大卖点, 因为它把所有重计算挪到了离线。- **帕累托前沿**: 一次跑出一整面光滑的前沿曲面, 管理者可**按需挑解** (人多挑低曝光的)。

优点: - 直击高利害考试的真实痛点 (可比性/成本/多目标), 思路新颖——把 CAT 从「在线逐题」搬到「离线整卷」。- 双层编码紧凑、有教育语义, 比 0-1 向量高效得多。- 在线零计算 + 一次出一组解, 工程上非常实用; 代码开源、可复现。

局限: - **自适应粒度粗**: 只有「试卷级、2 阶段、按 0.5 一刀切分难/易」, 远不如题目级 CAT 精细, 个性化程度打折 (作者也承认逐题选更个性化)。- **依赖稠密化**: 评测要先把数据稠密化, 稠密子集很小 (ASSIST 只剩 120 学生/83 题), 与真实大规模题库的差距存疑, 迁移到真实 GRE 量级题库还需验证。- **能力恒定假设**: 和大多数 CAT 一样默认能力测验中不变 (区别于 KT)。- AUC 公式书写存疑 ($TPR \cdot FPR$ 积分), 可能是笔误。- 未来工作: 动态依赖种群的曝光率指标、更多阶段。

对 KT/CAT 研究者的启发: 这篇给了一个「重新定义 CAT 范式」的范例——当你发现已有范式 (逐题在线) 在某个真实场景 (高利害考试) 下根本不适用时, 可以换一套**离线 + 群体优化**的框架绕过去。其「把多个考试质量因素显式建成 MOO 目标」的做法, 也能直接借鉴到你做 CAT 选题或 KT 评估时的多目标权衡上。需要注意它牺牲了自适应粒度换取可比性/低成本——这个 trade-off 是它的灵魂。